

FinAccrual ERP Platform

Database Configuration and Schema

MySQL / Sequelize data layer reference

Item	Detail
Project	FinAccrual ERP Platform (Backend API, Excel Add-in, Admin Dashboard)
Document	Database Configuration and Schema

This document describes how the FinAccrual backend connects to its database, how the schema is created and maintained, and the structure of every table.

1. Overview

The backend (Backend/Backend) stores all persistent data in a **MySQL** database accessed through the **Sequelize 6** ORM with the **mysql2** driver. The connection is created once in `src/core/database/index.js` and shared by every module.

Aspect	Value
Database engine	MySQL (Sequelize dialect: mysql, driver: mysql2)
ORM	Sequelize ^6.37.8
Connection file	src/core/database/index.js
Configuration source	src/core/config/index.js (values read from .env)
Schema management	sequelize.sync() on server start (no migration files)
SQL logging	Disabled (logging: false)
Number of tables	5 (users, admins, quickbooks_quickbookstoken, xero_tokens, notifications)

Note : The existing README.md and BACKEND_DOCUMENTATION.md mention SQLite. The running code uses MySQL only (dialect: 'mysql'). The sqlite3 and pg packages are listed in package.json but are not used by the database module.

2. Database Connection Configuration

2.1 Connection parameters

Connection values come from environment variables and fall back to the defaults below when a variable is missing.

Variable	Default in code	Purpose
DB_HOST	localhost	MySQL server host name or IP
DB_PORT	3306	MySQL server port
DB_NAME	quickbooks_xero	Database (schema) name.

		.env.example suggests finaccrual
DB_USER	root	MySQL user name
DB_PASSWORD	123456	MySQL password. Always override this outside local development
DB_SYNC_ALTER	false	When true, runs sync({ alter: true }) so Sequelize adjusts existing tables to match the models

2.2 How the connection is initialised

```
const sequelize = new Sequelize(config.DB.NAME, config.DB.USER,
config.DB.PASSWORD, {
  host: config.DB.HOST,
  port: config.DB.PORT,
  dialect: 'mysql',
  logging: false
});
```

On start-up src/server.js calls sequelize.sync({ alter: process.env.DB_SYNC_ALTER === 'true' }). When this succeeds the server logs Database synchronized., starts the hourly notification clean-up job and begins listening. If the database is unreachable the error Unable to connect to the database is logged and the HTTP server is **not** started.

2.3 Health check

GET /api/health calls sequelize.authenticate() and returns the database state.

```
{ "success": true, "server": "healthy", "instance": "server-1", "database":
"connected" }
```

3. Preparing the Database

Sequelize creates the tables but **not** the database itself. Create the database and a

dedicated user before the first start.

```
CREATE DATABASE finaccrual CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;
```

```
CREATE USER 'finaccrual'@'localhost' IDENTIFIED BY '<strong-password>';  
GRANT ALL PRIVILEGES ON finaccrual.* TO 'finaccrual'@'localhost';  
FLUSH PRIVILEGES;
```

Then set DB_NAME=finaccrual, DB_USER=finaccrual and DB_PASSWORD=<strong-password> in the backend .env file. The user needs CREATE, ALTER, INDEX, REFERENCES, SELECT, INSERT, UPDATE and DELETE privileges (covered by ALL PRIVILEGES on the schema).

4. Schema Management Strategy

- ☐ **No migrations.** The project has no migration folder. The models are the single source of truth for the schema.
- ☐ **First run:** sync() creates all missing tables. users is created first because the other tables reference it.
- ☐ **Model changes:** start once with DB_SYNC_ALTER=true so Sequelize alters the existing tables, then turn the flag back off (the server logs a reminder). Plain sync() never changes an existing table.
- ☐ **Production:** for a stable production database, prefer reviewed SQL scripts or a migration tool over alter: true.
- ☐ **Timestamps:** every table has created_at / updated_at (or createdAt / updatedAt for notifications), maintained by Sequelize.

5. Entity Relationships

Parent table	Child table	Foreign key	Cardinality
users	quickbooks_quickbookstoken	user_id → users.id	One user has many QuickBooks company connections

Parent table	Child table	Foreign key	Cardinality
users	quickbooks_quickbookstoken	user_id → users.id	One user has many QuickBooks company connections
users	xero_tokens	user_id → users.id	One user has many Xero organisation connections
users	notifications	userId → users.id	One user has many notifications
admins	(none)	—	Stand-alone table used only by the legacy admin login

Sequelize associations are declared in `src/core/database/index.js` as `User.hasMany(...)` and `<Model>.belongsTo(User, ...)` with the aliases `quickbooksTokens`, `xeroTokens` and `notifications`.

6. Table Definitions

6.1 users

Model: `src/modules/auth/user.model.js`. One row per person who signs in through Google or Microsoft. It also holds the current session tokens and the subscription plan.

Column	Type	Null	Default / Constraint	Description
id	VARCHAR(13)	No	PRIMARY KEY, immutable, generated	Public ID in the form FIN<year><5 random digits>, e.g. FIN202648213
name	VARCHAR(100)	No	Length 1–100	Display name from the identity

				provider
email	VARCHAR(150)	No	UNIQUE (users_email_unique), must be a valid e-mail	Lower-cased sign-in e-mail
provider	ENUM	No	'local' 'google' 'microsoft', default 'local'	Identity provider used for the account
google_id	VARCHAR(255)	Yes	NULL	Google account subject ID
microsoft_id	VARCHAR(255)	Yes	NULL	Microsoft Entra ID object ID
role	VARCHAR(50)	No	default 'user'; allowed 'user', 'admin'	Authorization role
is_active	TINYINT(1)	No	default 1	Set to 0 to block a user from all protected routes
plan	VARCHAR(50)	Yes	NULL	Subscription plan: trial, Basic, Standard, Pro (NULL until chosen)
access_token	TEXT	Yes	NULL	Last issued application JWT
access_token_expires_at	DATETIME	Yes	NULL	Expiry of the stored access token (1 hour)
refresh_token	TEXT	Yes	NULL	Opaque refresh token (128 hex)

				characters)
refresh_token_expire s_at	DATETIME	Yes	NULL	Expiry of the refresh token (7 days)
created_at	DATETIME	No	auto	Row creation time

Guard : A beforeUpdate hook throws an error if anyone attempts to change users.id after creation.

6.2 admins

Model: src/modules/admin/model.js. Legacy administrator accounts that sign in with e-mail and password (/api/admin/login).

Column	Type	Null	Default / Constraint	Description
id	INT	No	PRIMARY KEY, AUTO_INCREMENT	Admin ID
name	VARCHAR(100)	No	—	Admin name
email	VARCHAR(150)	No	UNIQUE	Login e-mail
password	VARCHAR(255)	No	—	bcrypt hash (cost factor 10) — never the plain password
created_at	DATETIME	No	auto	Row creation time
updated_at	DATETIME	No	auto	Last update time

6.3 quickbooks_quickbookstoken

Model: src/modules/quickbooks/model.js. One row per connected QuickBooks Online company.

Column	Type	Null	Default / Constraint	Description
realm_id	VARCHAR(50)	No	PRIMARY KEY	QuickBooks company (realm) ID

access_token	TEXT	No	—	QuickBooks access token (cleared to empty string on disconnect)
refresh_token	TEXT	No	—	QuickBooks refresh token
token_type	VARCHAR(50)	Yes	—	Token type, normally bearer
expires_in	INT	Yes	default 0	Absolute UNIX time (seconds) when the access token expires
x_refresh_token_expires_in	INT	Yes	default 0	Absolute UNIX time (seconds) when the refresh token expires (about 100 days)
session_info	TEXT	Yes	NULL	JSON snapshot of the session at connection time
user_id	VARCHAR(13)	No	FK → users.id	Owner of the connection
company_name	VARCHAR(255)	Yes	default	Display

	)		'QuickBooks Company'	name (user can rename)
status	VARCHAR(20)	Yes	default 'Not Synced'; allowed 'Not Synced', 'Active', 'Disconnected'	Connection state
last_synced_at	DATETIME	Yes	NULL	Time of the last successful master-data pull
record_count	INT	Yes	NULL	Cached total record count for the company
created_at / updated_at	DATETIME	No	auto	Row timestamps

6.4 xero_tokens

Model: [src/modules/xero/model.js](#). One row per connected Xero organisation (tenant). Uses **underscored: true**, so timestamps are **created_at** and **updated_at**.

Column	Type	Null	Default / Constraint	Description
tenant_id	VARCHAR(255)	No	PRIMARY KEY	Xero tenant (organisation) ID
access_token	TEXT	No	—	Xero access token
refresh_token	TEXT	No	—	Xero refresh token
expires_in	INT	No	—	Access-token lifetime / expiry value returned by Xero

token_type	VARCHAR(255)	No	—	Token type, normally Bearer
scope	TEXT	Yes	NULL	Granted OAuth scopes
session_info	TEXT	Yes	NULL	JSON snapshot of the session at connection time
user_id	VARCHAR(13)	No	FK → users.id	Owner of the connection
company_name	VARCHAR(255)	Yes	default 'Xero Organisation'	Display name (user can rename)
status	VARCHAR(20)	Yes	default 'Not Synced'; allowed 'Not Synced', 'Active', 'Disconnected'	Connection state
last_synced_at	DATETIME	Yes	NULL	Time of the last successful master-data pull
record_count	INT	Yes	NULL	Cached total record count
created_at / updated_at	DATETIME	No	auto	Row timestamps

6.5 notifications

Model: src/modules/notifications/notification.model.js. Per-user in-app notification history (success / error toasts) stored server-side so that each account sees only its own history.

Column	Type	Null	Default / Constraint	Description
id	CHAR(36)	No	PRIMARY KEY, UUID v4	Notification ID
userId	VARCHAR(13)	No	FK → users.id	Owning user
type	ENUM	No	'success' 'error'	Notification category
message	VARCHAR(500)	No	—	Main message

detail	VARCHAR(1000)	Yes	NULL	Secondary technical line
provider	ENUM	Yes	'quickbooks' 'xero' NULL	ERP the message relates to (NULL = generic)
read	TINYINT(1)	No	default 0	Read flag
createdAt / updatedAt	DATETIME	No	auto	Row timestamps (camelCase on this table)

- **Index:** composite index on (userId, createdAt) supports the "list my notifications, newest first" query.
- **Retention:** rows older than **24 hours** are hard-deleted by a clean-up job that runs once at start-up and then every hour.

7. Indexes and Constraints Summary

Table	Constraint / Index	Columns
users	PRIMARY KEY	id
users	UNIQUE users_email_unique	email
admins	PRIMARY KEY / UNIQUE	id / email
quickbooks_quickbookstoken	PRIMARY KEY, FOREIGN KEY	realm_id; user_id → users.id
xero_tokens	PRIMARY KEY, FOREIGN KEY	tenant_id; user_id → users.id
notifications	PRIMARY KEY, FOREIGN KEY, INDEX	id; userId → users.id; (userId, createdAt)

8. Data Rules and Lifecycle

Topic	Behaviour
Sign-up	First Google / Microsoft sign-in creates a users row with role = user, plan = NULL.
Existing e-mail	If the e-mail already exists, the provider ID (google_id /

	microsoft_id) is attached to the existing row.
Session tokens	Only one refresh token per user is valid; each refresh rotates it. Logout sets all four token columns to NULL.
ERP connect	A row is inserted into the QuickBooks / Xero table with status Not Synced.
First pull	After the first successful master-data pull, status becomes Active and last_synced_at is set.
ERP disconnect / revoke	Status becomes Disconnected; tokens and session_info are wiped.
Plan limits	Connections allowed per ERP: trial 1, Basic 1, Standard 3, Pro 10 (checked before OAuth starts).
Validation layers	Joi request schemas → hand-written checks → Sequelize model validators (e-mail format, allowed roles and statuses).

9. Reference DDL

The statements below are the MySQL equivalent of what Sequelize generates. They are provided for review and for teams that prefer to create the schema manually. Actual generated names and options may differ slightly.

```
CREATE TABLE users (
  id VARCHAR(13) NOT NULL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  provider ENUM('local','google','microsoft') NOT NULL DEFAULT 'local',
  google_id VARCHAR(255) NULL,
  microsoft_id VARCHAR(255) NULL,
  role VARCHAR(50) NOT NULL DEFAULT 'user',
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  plan VARCHAR(50) NULL,
  access_token TEXT NULL,
  access_token_expires_at DATETIME NULL,
  refresh_token TEXT NULL,
  refresh_token_expires_at DATETIME NULL,
```

```
created_at DATETIME NOT NULL,  
updated_at DATETIME NOT NULL,  
UNIQUE KEY users_email_unique (email)  
);  
  
CREATE TABLE admins (  
id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
name VARCHAR(100) NOT NULL,  
email VARCHAR(150) NOT NULL UNIQUE,  
password VARCHAR(255) NOT NULL,  
created_at DATETIME NOT NULL,  
updated_at DATETIME NOT NULL  
);  
  
CREATE TABLE quickbooks_quickbookstoken (  
realm_id VARCHAR(50) NOT NULL PRIMARY KEY,  
access_token TEXT NOT NULL,  
refresh_token TEXT NOT NULL,  
token_type VARCHAR(50),  
expires_in INT DEFAULT 0,  
x_refresh_token_expires_in INT DEFAULT 0,  
session_info TEXT,  
user_id VARCHAR(13) NOT NULL,  
company_name VARCHAR(255) DEFAULT 'QuickBooks Company',  
status VARCHAR(20) DEFAULT 'Not Synced',  
last_synced_at DATETIME NULL, record_count INT NULL,  
created_at DATETIME NOT NULL,  
updated_at DATETIME NOT NULL,  
FOREIGN KEY (user_id) REFERENCES users(id)  
);  
  
CREATE TABLE xero_tokens (  
tenant_id VARCHAR(255) NOT NULL PRIMARY KEY,
```

access_token TEXT NOT NULL,
 refresh_token TEXT NOT NULL,
 expires_in INT NOT NULL,
 token_type VARCHAR(255) NOT NULL,
 scope TEXT,
 session_info TEXT,
 user_id VARCHAR(13) NOT NULL,
 company_name VARCHAR(255) DEFAULT 'Xero Organisation',

10. Observations for the Team

The points below were found while reading the code. None of them stop the application from starting, but they are worth reviewing before a production release.

Sl.no	Observation	Suggested action
1	trial_ends_at is read and written by auth.service.js and user.repository.js, but the User model does not define this column. Sequelize ignores unknown fields on write, and listing users with that attribute (GET /api/admin/users) would fail with an "unknown column" error.	Add trial_ends_at DATETIME NULL to user.model.js (and run once with DB_SYNC_ALTER=true).
2	POST /api/auth/update-plan deletes connections with where: { mail: ... }, but the token tables have no mail column (they use user_id).	Change the filter to { user_id: req.user.userId }.
3	ERP access and refresh tokens are stored as plain text in the database.	Encrypt token columns at rest (application-level or database-level encryption).
4	realm_id / tenant_id is the only primary key, so a company can belong to only one FinAccrual user at a time.	Use a composite key (user_id, realm_id) if the same company must be shared.
5	expires_in columns are INT and store UNIX seconds; the signed 32-bit limit	Use BIGINT for these two columns.

	is reached in January 2038.	
6	Default fall-backs in config/index.js (root / 123456) are convenient for development only.	Always set explicit DB_* values in every non-local environment.
7	src/core/redis.js references config.REDIS and the redis package, neither of which exists in the project. The file is not imported anywhere.	Remove the file or add the configuration and dependency if Redis is planned.

